

## **Projeto – Feminicídio - A ação dos dados como um fato social.**

**Alexandre Machado Ferreira e Silva, Guilherme Henrique Souza, Marcus de Oliveira dos Santos, Pedro Henrique Levinski, Sara Belintani, Thiago Ximenes Naves,  
Orientador Alisson Rabelo Arantes**

PUC Minas

Curso de Tecnologia em Banco de Dados

[alexandre.silva.1551909@sga.pucminas.br](mailto:alexandre.silva.1551909@sga.pucminas.br),

[guilherme.souza.1456357@sga.pucminas.br](mailto:guilherme.souza.1456357@sga.pucminas.br),

[marcus.santos.441114@sga.pucminas.br](mailto:marcus.santos.441114@sga.pucminas.br),

[pedrolfilo@gmail.com](mailto:pedrolfilo@gmail.com),

[sara.belintani.contato@gmail.com](mailto:sara.belintani.contato@gmail.com),

[thiago.naves@sga.pucminas.br](mailto:thiago.naves@sga.pucminas.br).

O objetivo desta etapa é extrair os dados que estão em uma base de dados e inseri-los em algum SGBD relacional (SQL Server, MySQL, Oracle, PostgreSQL, ...) utilizando a linguagem SQL para posterior manipulação. Para isso, o grupo deve implementar uma pequena aplicação (programa) que leia o conteúdo de um arquivo texto no formato CSV e gere um script SQL para inserção dos dados no SGBD.

Para isso, criou-se uma pequena aplicação em C#, linguagem trabalhada pelo microfundamento algoritmos e lógica de programação. Este programa estabelece uma conexão com o Postgre, cria o banco de dados Feminicidio, suas tabelas e respectivas colunas. Descompacta o arquivo zipado do dataset escolhido gerando uma tabela temporária para armazenamento desses dados, bem como infere as colunas e tipo de conteúdo de cada arquivo, transformando no caso dos \*.parquet em .csv para melhor manipulação.

```

12  class Program
13  {
14      static string connectionString = "Host=localhost;Username=postgres;Password=2822;Database=
15
16      0 referências
17      static void Main(string[] args)
18      {
19          GarantirBanco();
20
21          string zipPath = @"C:\Users\marca\Downloads\archive (1).zip";
22          string extractPath = @"C:\Users\marca\Downloads\archive_extracted";
23
24          if (!Directory.Exists(extractPath))
25          {
26              ZipFile.ExtractToDirectory(zipPath, extractPath);
27              Console.WriteLine("ZIP extraído.");
28          }
29
30          foreach (var arquivo in Directory.GetFiles(extractPath))
31          {
32              string nomeTabela = Path.GetFileNameWithoutExtension(arquivo).ToLower();
33
34              if (arquivo.EndsWith(".csv", StringComparison.OrdinalIgnoreCase))
35              {
36                  CriarTabelaCsv(arquivo, nomeTabela);
37                  ProcessarCsvComCopy(arquivo, nomeTabela);
38              }
39              else if (arquivo.EndsWith(".parquet", StringComparison.OrdinalIgnoreCase))
40              {
41                  CriarTabelaParquet(arquivo, nomeTabela);
42                  ProcessarParquetComCopy(arquivo, nomeTabela);
43              }
44
45              ContarLinhas(nomeTabela);
46          }
47      }
48  }

```

\*Bloco 1 ( Main ) - Ilustração

Utilizou-se quatro fontes de códigos principais (bibliotecas externas) e de código aberto (Open Source): .NET(Runtime & Core) é a base da execução; Npgsql é o driver que permite a conexão entre o C# e o PostgreSQL; CsvHelper é a biblioteca usada para ler e mapear os arquivos CSV; ParquetSharp é o wrapper que permite ao C# ler os arquivos no formato Apache Parquet.

O registro dos dados é realizado através do comando COPY como solução para o INSERT sugerido na tarefa como forma de otimizar o tempo para o processamento dos arquivos de maior volume evitando a leitura de linha por linha por parte da aplicação.

O código foi dividido em blocos principais com um resumo do que fazem para melhor compreensão:

### Bloco 1 - Configuração e Ponto de Entrada (Main):

Este bloco define onde o banco de dados está e dita a regra de negócio: extrair o ZIP e processar cada arquivo encontrado. connectionString: Armazena as credenciais de acesso ao PostgreSQL (host, usuário, senha e banco).GarantirBanco(): Verifica se o banco "Feminicidio" existe; se não, ele o cria do zero.ZipFile.ExtractToDirectory: Descompacta o arquivo baixado para uma

pasta.local.foreach: Varre todos os arquivos da pasta e decide se chama a lógica de CSV ou Parquet baseada na extensão do arquivo.

## **Bloco 2 - Inferência de Tipos (InferirTipo e MapearTipo):**

Como arquivos CSV não dizem se uma coluna é número ou texto, este bloco "adivinha" o tipo de dado para criar a tabela corretamente no banco. TryParse: Tenta converter o texto para Inteiro, Double ou Data. Se conseguir, define o tipo SQL correspondente (INTEGER, DOUBLE PRECISION, DATE). Caso contrário, assume que é TEXT. No Parquet, o processo é mais direto, pois o arquivo Parquet já guarda metadados sobre o tipo físico de cada coluna (Int32, Float, etc.).

Trecho do código:

```
static string InferirTipo(string valor)
{
    if (int.TryParse(valor, out _)) return "INTEGER";
    if (double.TryParse(valor, NumberStyles.Any, CultureInfo.InvariantCulture, out _)) return "DOUBLE PRECISION";
    if (DateTime.TryParse(valor, out _)) return "DATE";
    return "TEXT";
}

// No caso do Parquet (MapearTipo):
return col.PhysicalType switch {
    PhysicalType.Int32 => "INTEGER",
    PhysicalType.Int64 => "BIGINT",
    PhysicalType.Double => "DOUBLE PRECISION",
    _ => "TEXT"
};
```

## **Bloco 3 - Criação Dinâmica de Tabelas (CriarTabela...):**

Antes de inserir os dados, o código precisa preparar a "caixa" (tabela) que vai recebê-los. csv.ReadHeader(): Lê a primeira linha do arquivo para dar nome às colunas da tabela. Take(10): Lê apenas as primeiras 10 linhas para inferir os tipos, evitando lentidão em arquivos gigantes. CREATE TABLE IF NOT EXISTS: Executa o comando SQL no PostgreSQL para criar a estrutura da tabela com os nomes e tipos descobertos.

Trecho do código:

```
csv.ReadHeader();

var colunas = csv.HeaderRecord;

var registros = csv.GetRecords<dynamic>().Take(10).ToList(); // Amostragem para tipos

string colunasSql = string.Join(" ", tipos.Select(kv => $"\"{kv.Key}\" \"{kv.Value}\""));

string comandoCreate = $"CREATE TABLE IF NOT EXISTS \"{nomeTabela}\" ({colunasSql});";

using var cmd = new NpgsqlCommand(comandoCreate, conn);

cmd.ExecuteNonQuery();
```

#### **Bloco 4 - Processamento e Carga em Massa (COPY):**

Este é o bloco de maior performance. Em vez de inserir linha por linha (o que seria lento), ele usa o protocolo `Binary Copy`. `Path.GetTempFileName()`, cria um arquivo temporário para padronizar os dados antes do envio. `csvReader.Read()`, percorre todas as linhas do arquivo original. `conn.BeginTextImport(...)` e inicia o comando `COPY` do PostgreSQL. É a forma mais rápida de injetar milhões de linhas em um banco de dados SQL. `File.Delete(tempCsv)`. Isso tudo feito, limpa os arquivos temporários para não lotar o HD do computador.

Trecho do código:

```
string tempCsv = Path.GetTempFileName(); // Arquivo temporário de padronização

while (csvReader.Read()) { /* percorre todas as linhas */ }

using var copy = conn.BeginTextImport($"COPY \"{nomeTabela}\" FROM STDIN (FORMAT csv, HEADER true);

using (var reader = new StreamReader(tempCsv))

{

    string linha;

    while ((linha = reader.ReadLine()) != null)

    {

        copy.WriteLine(linha); // Ingestão em massa

    }

}

File.Delete(tempCsv); // Limpeza de disco
```

## Bloco 5 - Auditoria (ContarLinhas):

Um pequeno bloco de controle de qualidade `SELECT COUNT(*)`. Após o processo, ele consulta o banco e exibe no console quantas linhas foram importadas com sucesso, permitindo conferir se o total bate com o arquivo original.

Trecho do código:

```
// Lógica interna simplificada do método ContarLinhas:

using var cmd = new NpgsqlCommand($"SELECT COUNT(*) FROM \"{nomeTabela}\"", conn);

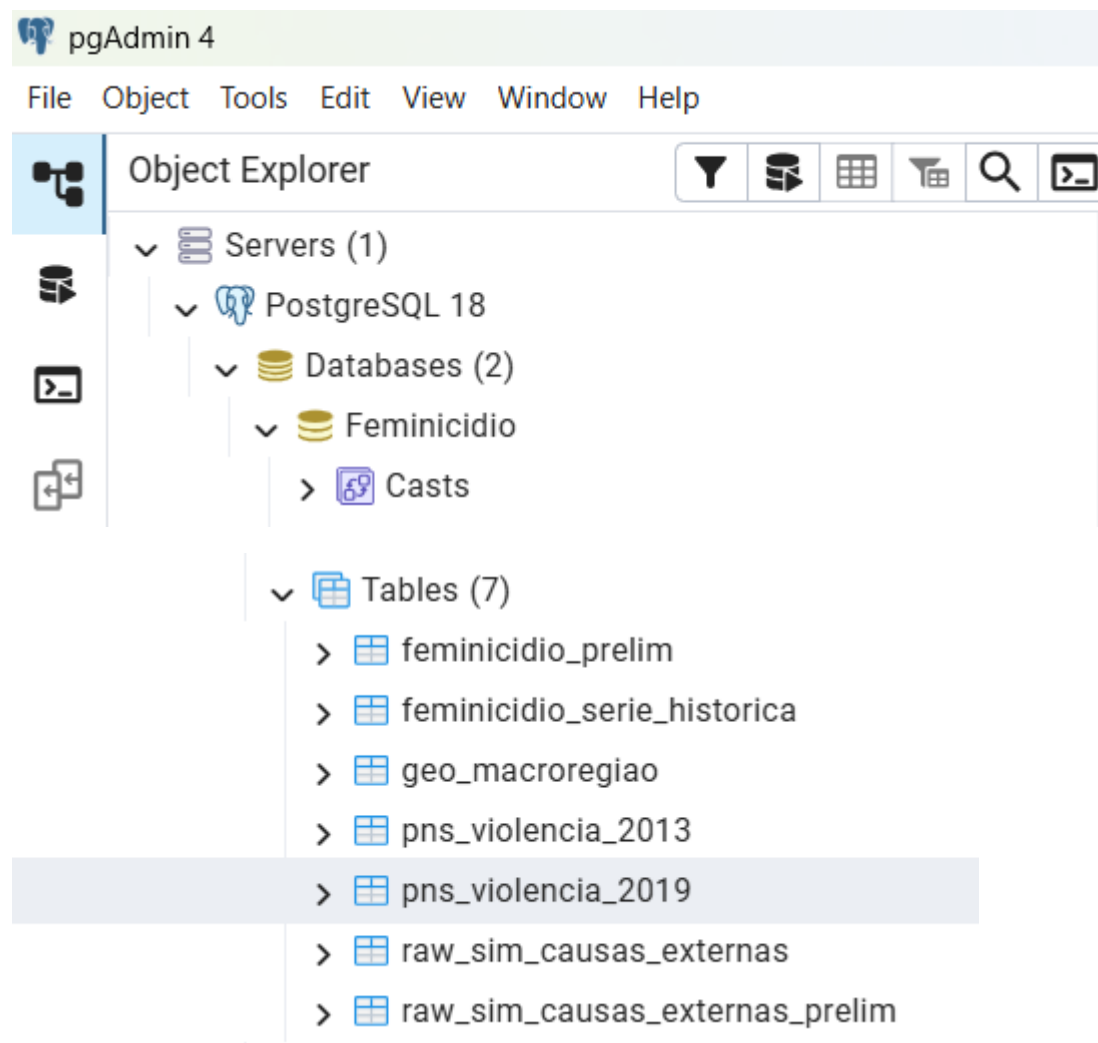
var total = cmd.ExecuteScalar();

Console.WriteLine($"Linhas importadas em {nomeTabela}: {total}");
```

## SCRIPTS SQL gerados pela aplicação

| Linha do Código | Comando SQL (Lógica)                                                 | O que ele faz no Banco de Dados                                            |
|-----------------|----------------------------------------------------------------------|----------------------------------------------------------------------------|
| 61              | <code>SELECT 1 FROM pg_database WHERE datname = 'Feminicidio'</code> | <b>Verifica</b> se o banco de dados já existe no servidor.                 |
| 64              | <code>CREATE DATABASE "Feminicidio"</code>                           | <b>Cria</b> o banco de dados físico para armazenar as tabelas.             |
| 105             | <code>CREATE TABLE IF NOT EXISTS "nome_tabela" (...)</code>          | <b>Cria a estrutura</b> da tabela (colunas e tipos) para arquivos CSV.     |
| 138             | <code>COPY "nome_tabela" FROM STDIN (FORMAT csv...)</code>           | <b>Importa</b> os dados em massa para dentro da tabela (Alta Performance). |
| 152             | <code>CREATE TABLE IF NOT EXISTS "nome_tabela" (...)</code>          | <b>Cria a estrutura</b> da tabela (colunas e tipos) para arquivos Parquet. |
| 158*            | <code>SELECT COUNT(*) FROM "nome_tabela"</code>                      | <b>Conta</b> o total de registros importados (Auditoria).                  |

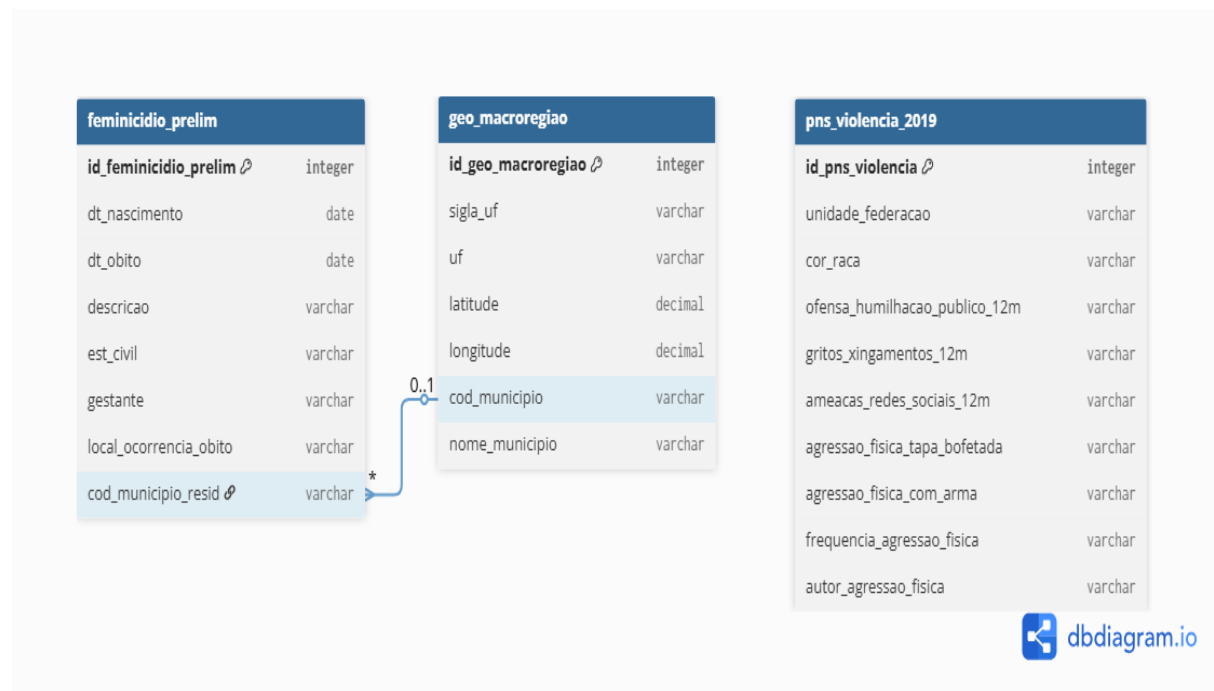
Imagem do banco de dados criado após aplicação no SGBD:



Amostra de tabela gerada:

|   | Unidade da Federação<br>text | Área<br>text | Sexo<br>text | Idade<br>integer | Cor ou raça<br>text | Ofensa/humilhação em público (12 meses)<br>text | Gritos/xingamentos (12 meses)<br>text |
|---|------------------------------|--------------|--------------|------------------|---------------------|-------------------------------------------------|---------------------------------------|
| 1 | Rondônia                     | Urba...      | Femini...    | 32               | Branca              | Não                                             | Não                                   |
| 2 | Rondônia                     | Urba...      | Femini...    | 64               | Parda               | Não                                             | Não                                   |
| 3 | Rondônia                     | Urba...      | Femini...    | 26               | Parda               | Sim                                             | Sim                                   |
| 4 | Rondônia                     | Urba...      | Femini...    | 41               | Preta               | Sim                                             | Não                                   |
| 5 | Rondônia                     | Urba...      | Femini...    | 38               | Preta               | Não                                             | Sim                                   |

Diagrama do banco com as colunas a serem utilizadas:



Imagens do código da aplicação:

```
105         tipo = InferirTipo(dict[columna].ToString());
106         break;
107     }
108     }
109     tipos[columna] = tipo;
110 }
111
112 string colunasSql = string.Join(", ", tipos.Select(kv => $"{kv.Key}\" {kv.Value}"));
113 string comandoCreate = $"CREATE TABLE IF NOT EXISTS \"{nomeTabela}\" ({colunasSql});";
114
115 using var conn = new NpgsqlConnection(connectionString);
116 conn.Open();
117 using var cmd = new NpgsqlCommand(comandoCreate, conn);
118 cmd.ExecuteNonQuery();
119
120 Console.WriteLine($"Tabela {nomeTabela} criada para CSV com tipos inferidos.");
121 }
122
123 1 referência
124 static void ProcessarCsvComCopy(string caminhoCsv, string nomeTabela)
125 {
126     string tempCsv = Path.GetTempFileName();
127     long linhasExportadas = 0;
128
129     var config = new CsvConfiguration(CultureInfo.InvariantCulture)
130     {
131         Delimiter = ";",
132         MissingFieldFound = null
133     };
134
135     using (var reader = new StreamReader(caminhoCsv))
136     using (var csvReader = new CsvReader(reader, config))
137     using (var writer = new StreamWriter(tempCsv))
138     using (var csvWriter = new CsvWriter(writer, CultureInfo.InvariantCulture))
139     {
140         csvReader.Read();
```

```

140 csvReader.ReadHeader();
141 var headers = csvReader.HeaderRecord;
142
143 foreach (var h in headers)
144     csvWriter.WriteField(h);
145 csvWriter.NextRecord();
146
147 while (csvReader.Read())
148 {
149     foreach (var h in headers)
150     {
151         var valor = csvReader.GetField(h);
152         csvWriter.WriteField(valor);
153     }
154     csvWriter.NextRecord();
155     linhasExportadas++;
156 }
157 }
158
159 Console.WriteLine($"Exportadas {linhasExportadas} linhas do CSV original para CSV temporário.");
160
161 using var conn = new NpgsqlConnection(connectionString);
162 conn.Open();
163 using var copy = conn.BeginTextImport($"COPY \"{nomeTabela}\" FROM STDIN (FORMAT csv, HEADER true);");
164 using (var reader = new StreamReader(tempCsv))
165 {
166     string linha;
167     while ((linha = reader.ReadLine()) != null)
168     {
169         copy.WriteLine(linha);
170     }
171 }
172
173 File.Delete(tempCsv);
174 Console.WriteLine($"CSV {caminhoCsv} importado via COPY para {nomeTabela}.");
175 }

```